Unit 3—Lesson 1:
Optionals

nil

```
struct Book {
  let name: String
  let publicationYear: Int
}

let firstHarryPotter = Book(name: "Harry Potter and the Sorcerer's Stone",
                            publicationYear: 1997)
let secondHarryPotter = Book(name: "Harry Potter and the Chamber of Secrets",
                             publicationYear: 1998)

let books = [firstHarryPotter, secondHarryPotter]
```

nil

```
let unannouncedBook = Book(name: "Rebels and Lions",
                           publicationYear: ???)
```

nil

```
let unannouncedBook = Book(name: "Rebels and Lions",
                           publicationYear: 0)
```

nil

```
let unannouncedBook = Book(name: "Rebels and Lions",
                           publicationYear: 2027)
```

nil

```
let unannouncedBook = Book(name: "Rebels and Lions",
                           publicationYear: nil)
```

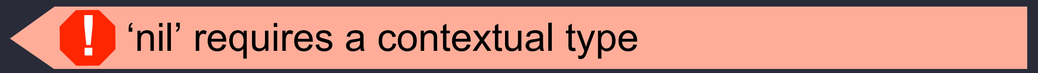
Nil is not compatible with expected argument type ‘Int’

```
struct Book {
  let name: String
  let publicationYear: Int?
}

let firstHarryPotter = Book(name: "Harry Potter and the Sorcerer's Stone",
                            publicationYear: 1997)
let secondHarryPotter = Book(name: "Harry Potter and the Chamber of Secrets",
                             publicationYear: 1998)

let books = [firstHarryPotter, secondHarryPotter]

let unannouncedBook = Book(name: "Rebels and Lions", publicationYear: nil)
```

Specifying the type of an optional

```
var serverResponseCode = 404
```

```
var serverResponseCode = nil
```

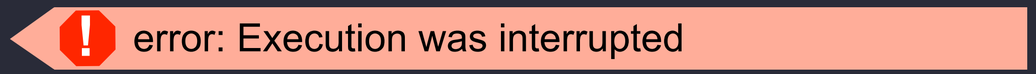



```
var serverResponseCode: Int? = 404
```

```
var serverResponseCode: Int? = nil
```

Working with optional values
Force-unwrap

```
if publicationYear != nil {
  let actualYear = publicationYear!
  print(actualYear)
}

let unwrappedYear = publicationYear!
```



Working with optional values
Optional binding

```
if let constantName = someOptional {
  //constantName has been safely unwrapped for use within the braces.
}
```

```
if let unwrappedPublicationYear = book.publicationYear {
  print("The book was published in \(unwrappedPublicationYear)")
}
else {
  print("The book does not have an official publication date.")
}
```

Functions and optionals
Return values

```
let string = "123"
let possibleNumber = Int(string)
```

```
let string = "Cynthia"
let possibleNumber = Int(string)
```

Functions and optionals
Defining

```
func printFullName(firstName: String, middleName: String?, lastName: String)
```

```
func textFromURL(url: URL) -> String?
```

Failable Initializers

Failable initializers in Swift are initializers that can return nil if the initialization process cannot be completed successfully. They are used when there are conditions that might prevent an instance of a class, structure, or enumeration from being created in a valid state.

Syntax: A failable initializer is defined by placing a question mark after the init keyword (init?).

It creates an optional instance of the type it is initializing. The return value is either a fully initialized instance or nil.

```
struct Toddler {
  var birthName: String
  var monthsOld: Int
}
```

Failable initializers

```
struct Toddler {
  var birthName: String
  var monthsOld: Int

  init?(birthName: String, monthsOld: Int) {
    if monthsOld < 12 || monthsOld > 36 {
      return nil
    } else {
      self.birthName = birthName
      self.monthsOld = monthsOld
    }
  }
}
```

```
struct Toddler {
  var birthName: String
  var monthsOld: Int

  init?(birthName: String, monthsOld: Int) {
    if birthName.isEmpty {
      return nil
    } else {
      self.birthName = birthName
      self.monthsOld = monthsOld
    }
  }
}
```

Failable initializers

```
init?(birthName: String, monthsOld: Int) {
    if birthName.isEmpty {
      return nil
    } else if monthsOld < 12 || monthsOld > 36{
        return nil
    }
      else {
      self.birthName = birthName
      self.monthsOld = monthsOld
    }
  }
```

Failable initializers

```
let myToddler = Toddler(birthName: "Todd", monthsOld: 40)
myToddler?.birthName
```

Returns Optional : It creates an optional instance of the type it is initializing. The return value is either a fully initialized instance or nil.

Failable initializers

```
let possibleToddler = Toddler(birthName: "Joanna", monthsOld: 14)
if let toddler = possibleToddler {
  print("\(toddler.birthName) is \(toddler.monthsOld) months old")
} else {
  print("The age you specified for the toddler is not between 1 and 3 yrs of age")
}
```

- The return type will be Toddler? so you need to unwrap with if/let (Optional Binding) or something.

Failable initializers

init! Failable Initializers

- Swift also offers a failable initializer that creates an implicitly unwrapped optional instance, denoted by init!.
- This can be used when the failure condition is highly unlikely, but if it occurs, it will cause a runtime crash.
- This is often used for interoperability with some Objective-C frameworks where Swift doesn't know if the initializer can return nil.
- In most pure Swift code, init? is preferred.

Optional chaining

Optional chaining is a safe and concise way in Swift to query and call properties, methods, and subscripts on an optional value that might be **nil**.

It allows you to access members without the risk of a runtime error if the optional value is empty.

Optional chaining uses a question mark (?) after an optional value. If the value is present, the operation continues; if it's nil, the chain returns nil.

This prevents runtime errors and the result is always an optional.

Multiple queries can be chained together, and the entire chain fails gracefully if any link in the chain is nil.

Optional chaining

```swift
struct Person {
  var age: Int
  var residence: Residence?
}

struct Residence {
  var address: Address?
}

struct Address {
  var buildingNumber: String?
  var streetName: String?
  var apartmentNumber: String?
}
```

```swift
// --- Scenario 1: Missing Data ---
let personA = Person(age: 25, residence: nil)

// The chain stops at 'residence' because it is nil
let numberA = personA.residence?.address?.buildingNumber

// --- Scenario 2: Complete Data ---
let address = Address(buildingNumber: "101", streetName: "Infinite Loop",
apartmentNumber: "4B")
let residence = Residence(address: address)
let personB = Person(age: 30, residence: residence)

// The chain succeeds through every level
let numberB = personB.residence?.address?.buildingNumber

// --- Printing Results ---
print("Result A: \(String(describing: numberA))")
print("Result B: \(String(describing: numberB))")
```

Optional chaining

```
let person = Person(age: 22)
if let theResidence = person.residence {
  if let theAddress = theResidence.address {
    if let theApartmentNumber = theAddress.apartmentNumber {
      print("He/she lives in apartment number \(theApartmentNumber).")
    }
  }
}
```

Optional chaining

```
if let theApartmentNumber = person.residence?.address?.apartmentNumber {
  print("He/she lives in apartment number \(theApartmentNumber).")
}
```

Implicitly Unwrapped Optionals

```
class ViewController: UIViewController {
  @IBOutlet var label: UILabel!
}
```

Unwraps automatically

Should only be used when need to initialize an object without supplying the value and you'll be giving the object a value soon afterwards

Nil-coalescing operator

```
5
```

g operator to check whether a optional contains a value or not.

- It is defined as (a ?? b).
- It unwraps an optional a and returns it if it contains a value, or returns a default value b if a is nil.

```
var someValue:Int!
let defaultValue = 5
let unwrappedValue:Int = someValue ?? defaultValue
print(unwrappedValue)
```

```
5
```

Guard statement

```
5
```

n with no if block.

- If the condition fails, else statement is executed. If not, next statement is executed.
- In given example, the guard contains a condition whether an optional someValue contains a value or not. If it contains a value then guard-let statement automatically unwraps the value and places the unwrapped value in temp constant. Otherwise, else block gets executed and and it would return to the calling function.

```
func testFunction() {
        let someValue:Int? = 5
        guard let temp = someValue else {
                return
        }
        print("It has some value \(temp)")
}

testFunction()
```

OUTPUT: It has some value 5

-

Using Guard Statement with Optionals

- The guard statement in Swift is used to safely unwrap optionals.
- It allows you to check for a condition and if that condition isn’t met, it exits the current scope, preventing the optional from being unwrapped.

```
func processData(data: Data?)
{
guard let data else {
 print("No data found")
 return
}
// process the data
}
```

Using Multiple Guard Statements

You can also use multiple guard statements for multiple optionals in the same scope. This can make your code more readable.

```
func processData(data: Data?, metadata: Metadata?)
{ guard let data else
{
print("No data found")
return
}
guard let metadata else
{
print("No metadata found")
return }
// process the data and metadata
}
```

Using Guard Statement with Optionals

In this example,

- the guard statement checks if the data variable is not nil.
- If data is nil, the guard statement triggers the else block and the function exits, preventing the code inside the function from executing.
- If data is not nil, the optional is unwrapped and the code inside the function continues to execute.

Unit 3, Lesson 1
Lab: Optionals.playground

Open and complete the exercises in Lab - Optionals.playground